C291 - Spring 25 - Systems Programming with C & Unix

Assignment 4

Q.1 Binomial Distribution (10 marks)

Write a function **float binomialDistribution**(**int k, int n, float p**) that returns the probability of k successes in n trials where p is the probability of success of an individual trial. The function should be called from main which will ask user to input these three parameters.
Hint: The probability can be computed using the probability mass function as given below:

$$f(k,n,p) = \binom{n}{k} p^k (1-p)^{n-k}$$

for $k = 0, 1, 2, \ldots, n$, where

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Example 1:
Input:
Enter k value: 4
Enter n value: 6
Enter p value: 0.3

Output:
The answer is 0.059535

Explanation:

$$f(4,6,0.3) = \binom{6}{4} 0.3^4 (1-0.3)^{6-4} = 0.059535.$$

Example 2:
Input:
Enter k value: 3
Enter n value: 5
Enter p value: 0.5

Output:
The answer is 0.31250

Note: The solution for this problem should be named as solution_1.c and it should be under C291-Spring-25/assignment4 where assignment4 would be the new directory you would create.

Q.2 Factorial (20 marks)

Write a C program that calculates the factorial of a positive integer entered by the user. In this program, you should use both a global variable and a static variable. Your program should have the following characteristics:

- Define a global variable named **ans** of type unsigned long long to store the factorial.
- Define a static variable named **static_count** of type int within the calculateFactorial function to keep track of how many times the calculateFactorial function is called.
- Create a function named **calculateFactorial** that takes an integer as an argument (let's call it n) and calculates the factorial of n. Inside this function, update the global variable **ans** to store the calculated factorial and increment the **static_count** static variable each time this function is called.

int calculateFactorial(int n)

- In the main function, prompt the user to enter a positive integer n.
- Use a for loop in the main function to call the calculateFactorial function iteratively. Each time you call the calculateFactorial function, store its return value (i.e., the value of **static_count**) in a local variable named **local_count**.
- After the for loop in the main function, print the calculated factorial value stored in the global variable **ans**.
- Also, print the number of times the calculateFactorial function was called, which is stored in the local variable **local_count** each time the function is called.
- Ensure that your program handles invalid input, specifically negative integers should be checked for and an appropriate error message or handling should be provided.

Example 1:

Input:

Enter a positive integer: 6

Output:

Factorial of 6 is 720
Number of times calculateFactorial was called: 6

Example 2:

Input:

Enter a positive integer: 15

Output:

Factorial of 15 is 1307674368000
Number of times calculateFactorial was called: 15

Example3:

Input:

Enter a positive integer: -3

Output:

Invalid input: Please enter a non-negative integer.

Note: The solution for this problem should be named as solution_2.c and it should be under C291-Spring-25/assignment4 where assignment4 would be the new directory you would create.

Q.3 Perfect Number (20 marks)

Write a C program to check whether the given number is a perfect number or not. [An integer number is said to be a Perfect Number if its factors, including 1(but not the number itself), sum to the number. E.g., 6 is a perfect number because 6=1+2+3]. You should take one number as input from the user and pass this value to a function called '**int isPerfectNumber(int n)'** which returns whether the number is perfect or not.

Example 1:
Input:
Enter a number: 6

Output:
6 is a perfect number

Example 2:
Input:
Enter a number: 50

Output:
50 is not a perfect number

Note: The solution for this problem should be named as solution_3.c and it should be under C291-Spring-25/assignment4 where assignment4 would be the new directory you would create.

Q.4 Craps Simulation (20 marks)

Simulate the game of craps with the following rules:

- A player rolls two dice. Each die has six faces. These faces contain 1, 2, 3, 4, 5, and 6 spots.
- After the dice have come to rest, the sum of the spots on the two upward faces is calculated.
- If the sum is 7 or 11 on the first throw, the player wins.
- If the sum is 2, 3, or 12 on the first throw (called "craps"), the player loses (i.e., the "house" wins).
- If the sum is 4, 5, 6, 8, 9, or 10 on the first throw, that sum becomes the player's "point."
- To win, you must continue rolling the dice until you "make your point." The player loses by rolling a 7 before making the point.

Note: you should create additional functions to roll dice, find sum of numbers on dice , check if player won, lost or scored a point.

```
e.g   int rollDice(void) {

//should return a random number between 1 and 6
}

int getSum(int a, int b) {
        //should return sum between 2 and 12
}

void checkCraps(int score) {

        // print won, craps or scored point based on rule above 3, 4, 5 and 6

}
```

Example 1:

Would you like to play a game of craps? Type y for yes or n for no: y
Dice is rolling ….
Score on dice are: 3 and 4
Sum is: 7
Congratulations you won!!!

Example 2:

Would you like to play a game of craps? Type y for yes or n for no : y
Dice is rolling …
Score on dice are: 1 and 1
Sum is: 2
Sorry you have lost and House won!!!

Example 3:

Would you like to play a game of craps? Type y for yes or n for no: y
Dice is rolling ….
Score on dice are: 2 and 2
Sum is: 4
Your point is 4 …
Dice is rolling …..
Score on dice are: 2 and 2
Sum is: 4
Consecutive same sum . you won!!

Example 3:

Would you like to play a game of craps? Type y for yes or n for no: y
Dice is rolling ….
Score on dice are: 2 and 2
Sum is: 4
Your point is 4 …
Dice is rolling …..
Score on dice are: 3 and 4
Sum is: 7
you lost!!

Note: The solution for this problem should be named as solution_4.c and it should be under C291-Spring-25/assignment4 where assignment4 would be the new directory you would create.

Q.5 Compute the Series (30 marks)

Write a C program to compute the series:

$$X + P_1 + \left(\frac{X^2}{2!}\right) + P_2 + \left(\frac{X^3}{3!}\right) + P_3 + \ldots + \left(\frac{X^n}{n!}\right) + P_n$$

where P1, P2, ..., Pn are prime numbers between two integers L and R. X and n are also integers.

Break the computation of this series into three components:
- Prime number generation
- Factorial computation
- Exponentiation

Write the following functions to implement the above:

float computeSeries(int X, int n, int L, int R): returns the series sum by calling the following three functions
int factorial(int n): returns the factorial of n
int generatePrime(int L, int R, int i): returns a prime number between L and R (Where i is the ith prime number between L and R)
int power(int X, int n): returns X^n

Note: If the total number of prime numbers between L and R is less than n, then set the rest of the Pi values to 0.
Note: The total sum of the series will be a `float` datatype. You don't have to format the decimal point to print it. It can be in any format.

Example 1:
Input:
Enter the value of X: 7
Enter the value of n: 5
Enter the value of L: 6
Enter the value of R: 13

Output:
Sum of series = 346.767

Hint:
Series will be

$$7 + 7 + \left(\frac{7^2}{2!}\right) + 11 + \left(\frac{7^3}{3!}\right) + 0 + \left(\frac{7^4}{4!}\right) + 0 + \left(\frac{7^n}{5!}\right) + 0$$

Where P_3, P_4, P_5 will be 0 because the total number of prime numbers between 6 and 13 are two which are less than 5(n)

Example 2:
Input:
Enter the value of X: 4
Enter the value of n: 3
Enter the value of L: 4
Enter the value of R: 20

Output:
Sum of series = 45.6667

Note: The solution for this problem should be named as solution_5.c and it should be under C291-Spring-25/assignment4 where assignment4 would be the new directory you would create.